

FYBCA Semester 2 — Complete Exam Notes

Subject 2: Introduction to Operating System

KBCNMU | CA-122 | Exam: 30 Marks | Complete Notes

TOPIC 1: Introduction to OS and Its Components

1.1 What is an Operating System?

An Operating System (OS) is system software that acts as an INTERFACE between the user and the computer hardware. It manages all computer resources and provides a platform for running application programs.

Without an OS, the user cannot interact with hardware. Examples: Windows, Linux, macOS, Android.

1.2 Definition

Definition: An Operating System is a program that controls the execution of application programs and acts as an interface between the user and computer hardware.

1.3 Functions of an OS

- Process Management — creates, schedules, and terminates processes.
- Memory Management — allocates and deallocates memory to processes.
- File System Management — organizes files on disk; read/write/delete.
- Device Management — manages I/O devices through device drivers.
- Security & Protection — controls access to resources.
- User Interface — provides CLI (Command Line) or GUI (Graphical).
- Error Detection — detects and responds to hardware/software errors.

1.4 Components of an OS

Component	Description
Kernel	Core of OS. Manages hardware directly. Always stays in memory.
Shell	Interface between user and kernel. Accepts user commands.
File System	Organizes data into files and directories on storage devices.
Device Drivers	Software that communicates between OS and hardware devices.
System Calls	Interface through which programs request OS services.
Process Manager	Handles process creation, scheduling, and termination.
Memory Manager	Handles allocation/deallocation of RAM to processes.

1.5 OS as a Resource Manager

The OS manages four main types of resources:

Resource	OS Role
----------	---------

Processor (CPU)	Allocates CPU time to different processes (scheduling).
Memory (RAM)	Decides which process gets how much memory.
I/O Devices	Controls access to keyboard, mouse, printer, disk, etc.
Files	Controls who can read, write, or execute files.

1.6 Exam Questions

Questions

- Define an operating system. What are its main functions? (6 marks)
- What are the components of an OS? Explain each. (6 marks)
- How does an OS act as a resource manager? (4 marks)

TOPIC 2: OS Operations and Security

2.1 Dual-Mode Operation

To protect the OS from user programs, modern CPUs operate in two modes:

Mode	Also Called	What Runs Here	Privileges
User Mode	Unprivileged Mode	User applications	Limited — cannot access hardware directly
Kernel Mode	Privileged / Supervisor Mode	OS kernel	Full — can execute any instruction

When user program needs OS service, it makes a SYSTEM CALL — switches from user mode to kernel mode temporarily.

2.2 Timer

The OS uses a TIMER to prevent a program from running forever (infinite loop). If a process runs beyond its time limit, the timer generates an interrupt, and the OS takes control back.

2.3 OS Security

- Authentication — verifying identity of users (passwords, biometrics).
- Authorization — controlling what authenticated users can access.
- Encryption — encoding data so only authorized users can read it.
- Access Control Lists (ACL) — defines who can access which files.
- Audit/Logging — recording system events to detect intrusions.
- Firewall — controls network traffic to protect the system.

2.4 Protection vs Security

Protection	Security
Controls access to resources within the system	Defends system against external threats
Internal mechanism	External defense mechanism
Example: file permissions	Example: firewall, antivirus
Important Points • Dual-mode operation protects the OS kernel from user programs. • Kernel mode has full hardware access; user mode is restricted. • System call switches from user mode to kernel mode. • Timer prevents monopolization of CPU by a single process.	

TOPIC 3: Abstract View of OS — User View & System View

3.1 User View of OS

The user sees the OS through the INTERFACE it provides. Different users have different views:

User Type	View of OS
PC User	Wants ease-of-use, GUI, fast response. OS focuses on user convenience.
Mainframe User	Shares system with many users. OS focuses on resource utilization.
Workstation User	Mix of personal use and resource sharing with network servers.
Mobile User	Battery life, small screen, touch input. OS optimized for mobile hardware.
Embedded System User	No user interface. OS runs devices like washing machines, cars.

3.2 System View of OS

From the system's perspective, the OS is a RESOURCE ALLOCATOR and CONTROL PROGRAM:

- Resource Allocator: Decides how to allocate CPU, memory, I/O to competing processes.
- Control Program: Controls execution of user programs to prevent errors and misuse.

3.3 OS as a Layer Model

Layer	Description
Hardware	Physical components — CPU, RAM, disk, I/O devices.
Kernel	Core OS — directly manages hardware.
System Programs	Utilities like compiler, assembler, shell.
Application Programs	User applications — browsers, word processors.
Users	End users who interact with the system.

Key Points

- User view: ease of use, performance, convenience.
- System view: resource management and control.
- OS hides hardware complexity from users.
- OS provides uniform interface to diverse hardware.

TOPIC 4: OS Services and System Calls

4.1 OS Services

The OS provides various services to users and programs:

Service	Description
Program Execution	Load program into memory, run it, handle normal/abnormal termination.
I/O Operations	Provides interface for reading/writing files and devices.
File System Manipulation	Create, delete, read, write, and search files and directories.
Communications	Processes exchange information via shared memory or message passing.
Error Detection	Detect and correct errors in CPU, memory, I/O, user programs.
Resource Allocation	Allocate CPU time, memory, I/O devices to multiple users/jobs.
Accounting	Track resource usage per user for billing/optimization.
Protection & Security	Control access to resources; protect from unauthorized access.

4.2 System Calls

A SYSTEM CALL is the mechanism through which a running program requests a service from the OS kernel. It is the programming interface between user programs and the OS.

When a system call is made: 1) User program makes a system call. 2) CPU switches from User Mode to Kernel Mode. 3) OS performs the requested service. 4) CPU switches back to User Mode.

4.3 Types of System Calls

Category	Examples	Description
Process Control	fork(), exit(), wait()	Create, terminate, and wait for processes.
File Management	open(), read(), write(), close()	Operations on files.
Device Management	ioctl(), read(), write()	Control and communicate with devices.
Information Maintenance	getpid(), alarm(), sleep()	Get/set system info, time, date.
Communications	pipe(), socket(), send(), recv()	Interprocess communication.
Protection	chmod(), chown(), umask()	Control resource access permissions.

4.4 Example: System Call in C

```
// Example: read() system call in Unix/Linux
#include <unistd.h>
int main() {
    char buf[100];
    read(0, buf, 100);    // 0=stdin, read 100 bytes
    write(1, buf, 100);   // 1=stdout, write 100 bytes
    return 0;
}
```

Important Points

- System calls provide the interface between user programs and OS.
- System call switches CPU from user mode to kernel mode.
- In Linux, system calls have numbers (e.g., read=0, write=1).
- fork() creates a new (child) process.
- exit() terminates the current process.
- Common system calls: open, read, write, close, fork, exec, wait, exit.

Exam Questions

- What is a system call? Explain its types with examples. (6 marks)
- What services does an OS provide? Explain. (6 marks)
- What happens when a system call is made? (4 marks)

TOPIC 5: Computer System Architecture

5.1 Single Processor System

A single-processor system has ONE main CPU that executes a general-purpose instruction set. It may have special-purpose processors (disk, keyboard controllers), but only one main CPU handles all computation.

- Simple design and lower cost.
- Only one process can run at a time (no true parallelism).
- Examples: Old PCs, embedded systems.

5.2 Multiprocessor Systems

Multiple processors share the computer bus, clock, memory, and I/O devices. Also called PARALLEL SYSTEMS or TIGHTLY COUPLED SYSTEMS.

Advantages of Multiprocessor Systems

- Increased Throughput: More processes completed per unit time.
- Economy of Scale: Cheaper than multiple single-processor systems.
- Increased Reliability: If one processor fails, others continue.

Types of Multiprocessing

Type	Description
Symmetric Multiprocessing (SMP)	Each processor runs identical copy of OS; processors share memory. Most common.
Asymmetric Multiprocessing (AMP)	One master processor controls the system; others execute tasks assigned by master.

5.3 Multi-Core Systems

Multiple processing cores on a single chip. Faster communication between cores since they share on-chip cache. More energy-efficient than multiple chips.

5.4 Clustered Systems

Multiple systems (nodes) connected via high-speed LAN that cooperate to accomplish a computational task. Provides HIGH AVAILABILITY — if one node fails, another takes over.

Type	Description
Asymmetric Clustering	One standby machine monitors the active server. If active fails, standby takes over.
Symmetric Clustering	Multiple nodes run applications, monitor each other. More efficient.

Important Points

- Multiprocessor = multiple CPUs; Multi-core = multiple cores on ONE chip.
- SMP: All processors are equal; AMP: Master-slave arrangement.
- Clustered systems provide high availability and fault tolerance.
- NUMA (Non-Uniform Memory Access): Each CPU has local memory; faster access to local memory.

Exam Questions

- Differentiate between single-processor and multiprocessor systems. (4 marks)
- What is symmetric multiprocessing? How is it different from asymmetric? (4 marks)
- What are clustered systems? Explain their types. (4 marks)

TOPIC 6: Types of Operating Systems

6.1 Batch Operating System

Jobs with similar requirements are grouped (batched) together and processed without user interaction. A special operator or batch monitor submits jobs to CPU one by one.

- No direct interaction between user and system during execution.
- Jobs are collected, queued, and processed in batch.
- Examples: Early IBM systems, payroll processing.
- Disadvantages: Starvation, long waiting time, difficult debugging.

6.2 Time-Sharing (Multitasking) OS

Multiple users can use the system simultaneously. CPU time is divided into small TIME SLICES (also called QUANTUM). Each user gets a time slice in round-robin fashion.

- Gives illusion that each user has dedicated computer.
- Response time is small (typically < 1 second).
- Examples: Unix, Linux, Windows.
- Disadvantage: Security concern; many users sharing system.

6.3 Real-Time Operating System (RTOS)

Must respond to inputs within a STRICT TIME DEADLINE. Used in systems where timing is critical.

Type	Description	Examples
Hard Real-Time	Must complete within EXACT deadline. Missing deadline = system failure.	Airbag systems, nuclear reactors, medical devices
Soft Real-Time	Deadline is preferred but not strict. Performance degrades if missed.	Video streaming, online transactions

6.4 Distributed Operating System

Multiple independent computers connected via network that appear as ONE system to users. No single computer has complete OS — the OS is distributed across all machines.

- Data and programs spread across multiple machines.
- Provides resource sharing, reliability, and scalability.
- Examples: Google's distributed system, Hadoop clusters.

6.5 Comparison of OS Types

OS Type	User Interaction	Response Time	Example Use
Batch OS	No interaction during job	High (minutes/hours)	Payroll, billing
Time-Sharing OS	Multiple users, interactive	Low (seconds)	Unix, Linux
Real-Time OS	Immediate response required	Very low (milliseconds)	Medical devices, robotics
Distributed OS	Across network	Variable	Cloud computing

Important Points

- Batch OS: No interaction; jobs batched and processed sequentially.

- Time-sharing: Multiple users share CPU using time slices/quantum.
- Hard RTOS: Strict deadline — must complete on time.
- Soft RTOS: Preferred deadline — degraded performance if missed.
- Distributed OS: Multiple computers appear as ONE system.

Exam Questions

- What is a batch operating system? What are its advantages and disadvantages? (4 marks)
- Explain time-sharing operating system with example. (4 marks)
- Differentiate between hard real-time and soft real-time OS. (4 marks)
- What is a distributed operating system? (4 marks)

TOPIC 7: Process Management

7.1 What is a Process?

A PROCESS is a program in execution. A program is passive (stored on disk); a process is active (loaded in memory and running). A process includes the program code, current activity (registers, program counter), and its data.

7.2 Process States

State	Description
New	Process is being created.
Ready	Process is waiting to be assigned to the processor.
Running	Instructions are being executed by the CPU.
Waiting/Blocked	Process is waiting for some event (I/O completion).
Terminated	Process has finished execution.

State transitions: New → Ready → Running → (Waiting or Terminated). Running → Ready if time quantum expires.

7.3 Process Control Block (PCB)

The OS maintains a data structure called PCB (Process Control Block) for each process. It contains all information about the process.

PCB Field	Contents
Process State	Current state: new, ready, running, waiting, terminated.
Process ID (PID)	Unique number identifying the process.
Program Counter	Address of next instruction to execute.
CPU Registers	Contents of all process-specific registers.
Memory Management Info	Base/limit registers, page tables.
I/O Status	List of I/O devices allocated to process, open files.
Accounting Info	CPU time used, real time used, process priority.

7.4 CPU Scheduling

When multiple processes are ready, the OS must decide which process gets the CPU next. This is done by the CPU Scheduler.

Scheduling Algorithm	Description
FCFS (First Come First Served)	Process that arrives first gets CPU first. Simple but may cause long wait.
SJF (Shortest Job First)	Process with shortest burst time gets CPU first. Optimal but needs future knowledge.

Round Robin	Each process gets a fixed time quantum in circular order. Best for time-sharing.
Priority Scheduling	Process with highest priority gets CPU first.

7.5 Threads

A THREAD is the smallest unit of CPU execution. A process can have multiple threads. All threads of a process share the same memory space (code, data, heap) but have separate stacks and registers.

Process	Thread
Heavyweight process	Lightweight process
Own separate memory space	Shares memory with parent process
Slow to create and switch	Fast to create and switch
Inter-process communication is complex	Communication between threads is easy

Important Points

- Process = program in execution.
- Process states: New, Ready, Running, Waiting, Terminated.
- PCB stores all information about a process.
- CPU Scheduler selects which process runs next.
- Round Robin is most commonly used in time-sharing systems.
- Thread is lighter than process; multiple threads share same memory.

Exam Questions

- What is a process? Explain process states with diagram description. (6 marks)
- What is PCB? What information does it contain? (4 marks)
- Explain CPU scheduling algorithms. (6 marks)
- Differentiate between process and thread. (4 marks)

TOPIC 8: Linux Basics and Architecture

8.1 What is Linux?

Linux is a FREE, OPEN-SOURCE, multi-user, multi-tasking operating system based on Unix. Created by Linus Torvalds in 1991. It is widely used in servers, supercomputers, Android (Linux kernel), and embedded systems.

8.2 Features of Linux

- Open Source: Source code is freely available.
- Multi-user: Multiple users can use the system simultaneously.
- Multi-tasking: Multiple processes run simultaneously.
- Portability: Can run on various hardware platforms.
- Security: Strong file permissions, user authentication.
- Stability: Rarely crashes; runs for years without rebooting.
- Free: No licensing cost.

8.3 Linux Architecture

Layer	Description
Hardware	Physical components: CPU, RAM, hard disk, network card.
Kernel	Core of Linux OS. Manages hardware, processes, memory, files.
Shell	Command interpreter. Accepts user commands and passes to kernel.
System Libraries	Functions that applications use to interact with kernel.
System Utilities	Programs for system administration (ls, cp, grep etc.).
Application Programs	User-level programs: browsers, editors, games.

8.4 Types of Linux Shells

Shell	Full Name	Features
sh	Bourne Shell	Original Unix shell; basic features.
bash	Bourne Again Shell	Most popular; enhanced sh; default in most Linux.
csh	C Shell	Syntax like C language.
ksh	Korn Shell	Combines features of sh and csh.
zsh	Z Shell	Extended bash with more features.

8.5 Kernel Functions

- Process Management: create, schedule, terminate processes.
- Memory Management: allocate/deallocate memory.

- Device Management: communicate with hardware via drivers.
- File System Management: read, write, organize files.
- System Calls: interface between user programs and kernel.

Important Points

- Linux is free, open-source, based on Unix kernel.
- Linus Torvalds created Linux kernel in 1991.
- Architecture layers: Hardware > Kernel > Shell > Applications.
- Kernel is the heart of Linux OS — always stays in memory.
- bash (Bourne Again Shell) is the default shell in most Linux distributions.

Exam Questions

- What is Linux? What are its features? (4 marks)
- Explain the architecture of Linux with diagram. (6 marks)
- What is a shell? What are different types of shells in Linux? (4 marks)
- What are the functions of the Linux kernel? (4 marks)

TOPIC 9: Linux Commands — Complete Reference

9.1 File and Directory Commands

Command	Syntax	Description	Example
ls	ls [options] [dir]	List files and directories	ls -la /home
pwd	pwd	Print working (current) directory	pwd
cd	cd [directory]	Change directory	cd /home/user
mkdir	mkdir dirname	Make new directory	mkdir myFolder
rmdir	rmdir dirname	Remove empty directory	rmdir myFolder
touch	touch filename	Create empty file	touch file.txt
cp	cp source dest	Copy file/directory	cp a.txt b.txt
mv	mv source dest	Move or rename file	mv old.txt new.txt
rm	rm [options] file	Remove/delete file	rm -rf mydir
cat	cat filename	Display file contents	cat file.txt
more	more filename	Display file page by page	more bigfile.txt
less	less filename	Like more but can scroll back	less bigfile.txt
find	find path -name	Search for files	find / -name file.txt
chmod	chmod mode file	Change file permissions	chmod 755 script.sh
chown	chown user file	Change file owner	chown root file.txt

9.2 Understanding chmod Permissions

File permissions are represented as three sets for: Owner | Group | Others

Permission	Symbol	Octal Value
Read	r	4
Write	w	2
Execute	x	1
No permission	-	0

Example: chmod 755 file.txt means: Owner=7(rwx), Group=5(r-x), Others=5(r-x)

9.3 Process Commands

Command	Description	Example
ps	Show running processes	ps aux
top	Real-time process monitor	top
kill	Kill a process by PID	kill 1234
bg	Move process to background	bg %1
fg	Move process to foreground	fg %1

9.4 Filter Commands

Command	Description	Example
head	Display first N lines of a file (default: 10)	head -5 file.txt
tail	Display last N lines of a file (default: 10)	tail -10 file.txt
sort	Sort lines of a file alphabetically/numerically	sort file.txt
grep	Search for pattern in file	grep 'hello' file.txt
wc	Word count (lines, words, chars)	wc -l file.txt
cut	Cut specific columns from file	cut -d: -f1 /etc/passwd
uniq	Remove duplicate adjacent lines	sort file.txt uniq

9.5 grep Options

Option	Description	Example
-i	Case-insensitive search	grep -i 'linux' file.txt
-v	Show lines NOT matching pattern	grep -v 'error' log.txt
-n	Show line numbers	grep -n 'hello' file.txt
-r	Recursive search in directories	grep -r 'main' /src/
-c	Count matching lines	grep -c 'error' log.txt

9.6 VI Editor

vi (Visual Editor) is the standard text editor in Linux/Unix. It has THREE modes:

Mode	How to Enter	What You Can Do
Command Mode	Default mode when vi opens	Navigate, delete, copy, paste. Press : for commands.
Insert Mode	Press i, a, o, I, A, O	Type and edit text.
Last Line Mode	Press : from command mode	Save, quit, search, replace.

9.7 Common VI Commands

Command	Action
i	Insert before cursor
a	Append after cursor
o	Open new line below
x	Delete character at cursor
dd	Delete entire current line
yy	Copy (yank) current line
p	Paste copied/deleted text
u	Undo last change
:w	Save file
:q	Quit vi
:wq	Save and quit
:q!	Quit without saving
:/pattern	Search for pattern
:s/old/new/	Replace first occurrence
:s/old/new/g	Replace all occurrences in line

Important Points

- ls -l shows long listing with permissions, owner, size, date.
- rm -rf recursively and forcefully deletes directory.
- chmod 777 gives full permissions to everyone (dangerous).
- grep is case-sensitive by default; use -i for case-insensitive.
- head and tail default to 10 lines; use -n to specify count.
- vi opens in Command Mode; press i to switch to Insert Mode.
- Always save in vi: :wq to save and quit; :q! to quit without saving.

TOPIC 10: Shell Programming

10.1 What is a Shell Script?

A shell script is a text file containing a sequence of Linux commands. It allows automation of repetitive tasks. Shell scripts are executed by the shell program (usually bash).

10.2 Creating and Running a Shell Script

```
# Step 1: Create script file
vi myscript.sh

# Step 2: First line must be shebang line
#!/bin/bash          <- This tells OS which shell to use

# Step 3: Make it executable
chmod +x myscript.sh

# Step 4: Run the script
./myscript.sh
```

10.3 Variables in Shell

```
#!/bin/bash

# Assigning variables (NO spaces around = sign)
name="Vishal"
age=20

# Displaying variables (use $ to access value)
echo "Name: $name"
echo "Age: $age"

# Read user input
echo "Enter your city:"
read city
echo "You live in: $city"
```

10.4 if-elif-else Statement

```
#!/bin/bash

echo "Enter a number:"
read num

if [ $num -gt 0 ]
then
    echo "Positive number"
elif [ $num -lt 0 ]
then
    echo "Negative number"
else
    echo "Zero"
fi
```

Comparison operators in shell scripts:

Operator	Meaning	Example
----------	---------	---------

-eq	Equal to	[\$a -eq \$b]
-ne	Not equal to	[\$a -ne \$b]
-gt	Greater than	[\$a -gt \$b]
-lt	Less than	[\$a -lt \$b]
-ge	Greater than or equal	[\$a -ge \$b]
-le	Less than or equal	[\$a -le \$b]

10.5 for Loop

```
#!/bin/bash

# Print 1 to 5
for i in 1 2 3 4 5
do
    echo "Number: $i"
done

# Loop through files
for file in *.txt
do
    echo "File: $file"
done
```

10.6 while Loop

```
#!/bin/bash

count=1
while [ $count -le 5 ]
do
    echo "Count: $count"
    count=$((count + 1))    # increment
done
```

10.7 until Loop

```
#!/bin/bash
# until loop runs while condition is FALSE (opposite of while)

n=1
until [ $n -gt 5 ]
do
    echo "Number: $n"
    n=$((n + 1))
done
```

10.8 Functions in Shell

```
#!/bin/bash

# Defining a function
greet() {
    echo "Hello, $1!"      # $1 is the first argument
    echo "Welcome to Linux Shell Programming"
```

```

}

# Calling the function
greet "Vishal"
greet "Rahul"

```

10.9 echo and read Commands

Command	Description	Example
echo	Print text to screen	echo "Hello World"
echo -n	Print without newline	echo -n "Enter name: "
echo -e	Enable escape sequences	echo -e "Line1\nLine2"
read	Read input from user	read varName
read -p	Prompt and read	read -p "Enter name: " name

10.10 Command Line Arguments

```

#!/bin/bash
# $0 = script name, $1 $2... = arguments, $# = count, $@ = all args

echo "Script name: $0"
echo "First argument: $1"
echo "Second argument: $2"
echo "Total arguments: $# "
echo "All arguments: $@"

```

Must Remember

- First line of shell script: `#!/bin/bash` (shebang line).
- No spaces around `=` in variable assignment: `x=5` (correct), `x = 5` (wrong).
- Access variable value with `$` prefix: `$varName`.
- `if` block ends with `fi`; `for/while/until` loops end with `done`.
- Arithmetic: `$((expression))` e.g., `$((a + b))`.
- String comparison: `==` `!=` `-z` (empty) `-n` (not empty).
- Make script executable: `chmod +x script.sh`
- Run script: `./script.sh` or `bash script.sh`

Exam Questions

- Write a shell script to read two numbers and print their sum. (4 marks)
- Write a shell script to check if a number is even or odd. (4 marks)
- Write a shell script to print multiplication table of a given number. (4 marks)
- Explain for loop in shell programming with example. (4 marks)
- What are shell variables? Explain with example. (4 marks)
- Write a shell script to find the largest of three numbers. (6 marks)

QUICK REVISION SUMMARY — Subject 2: OS

Topic	Key Points
OS Definition	Interface between user and hardware. Manages resources: CPU, memory, I/O, files.
OS Components	Kernel, Shell, File System, Device Drivers, System Calls, Process Manager, Memory Manager.
Dual Mode	User Mode (limited) and Kernel Mode (full access). System call switches between modes.
OS Types	Batch (no interaction), Time-sharing (multiple users, quantum), Real-Time (strict deadlines), Distributed.
System Calls	Interface between programs and OS kernel. Types: Process, File, Device, Info, Communication, Protection.
Architecture	Single vs Multi-processor. SMP=equal processors, AMP=master-slave. Clustered=high availability.
Process	Program in execution. States: New, Ready, Running, Waiting, Terminated. PCB stores all info.
Linux	Free, open-source, multi-user. Layers: Hardware > Kernel > Shell > Applications.
Linux Commands	ls, cd, mkdir, cp, mv, rm, chmod, grep, head, tail, sort, cat, find.
chmod	r=4, w=2, x=1. chmod 755: Owner=rwx, Group=r-x, Others=r-x.
VI Editor	3 modes: Command, Insert (press i), Last Line (press :). :wq=save+quit, :q!=quit no save.
Shell Script	#!/bin/bash. Variables: x=5, \$x. if-fi, for-done, while-done. read for input. echo for output.